



FRONTEND ENGINEERS • UX CONTRIBUTORS

UI MANUAL

Chart, editor, UI shell, indicators, results, store

Open charting PWA — own the axes, swap the engine



CONTENTS

01	Overview
02	Charting
03	Editor
04	Debugging
05	Ui Shell
06	Indicators
07	Results And Strategy
08	Store

OVERVIEW

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED

THIS FILE IS PART OF PYNESCRYPT.

PYNESCRYPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY

IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY

THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR

(AT YOUR OPTION) ANY LATER VERSION.

PYNESCRYPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL, BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF

MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.

YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE

ALONG WITH PYNESCRYPT. IF NOT, SEE

[HTTPS://WWW.GNU.ORG/LICENSES/](https://www.gnu.org/licenses/).

SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "UI" description: "AXIS UI subsystem: chart host, editor, UI shell, indicators, results, and Solid store—presentation without a closed engine."

UI

UI is the AXIS browser presentation layer: everything the operator sees and clicks that is *not* language evaluation. Engines are plugins; AXIS applies their outputs to panes, markers, and drawers.



Abstract

Subsystem	Responsibility
Charting	Panes, multi-chart, replay, compare, drawings, badges
Editor	CM6, diagnostics, ruler, git bar, stats, dock/popout
Debugging	Scriptlogs, Profiler, Debug chips, Pins, Problems
UI shell	Topbar groups, docks side-by-side, palette, shortcuts, mobile shell, alerts
Scripts	Run pipeline, Scripts panel, overlay/sub-pane routing
Results & strategy	Event normalize, trades, equity, export
Store	Solid store, persistence, active plugins

Conceptual model

```
flowchart TB
    SHELL[UI shell] --> STORE[Solid store]
    ED[Editor] --> STORE
    ED -->|Run| IND[indicators/runner]
    IND --> ENG[Engine plugin]
    ENG --> IND
    IND --> CH[Chart host]
    IND --> RES[Results]
    STORE --> CH
    STORE --> SHELL
```

Invariant (AXIS ≠ engine): no Solid component imports PYNE directly; only engine modules talk to Python/HTTP runtimes.

Interface surface (visual map)

```
┌ Topbar (grouped: market · data · compute · layout · panels · system) ┐
├ left docks ┌ ChartHost (1-4 slots) + drawings ┐ right docks ───┐
│ watchlist │ multi-pane + badges + replay │ indicators|editor │
│ layers ... │ │ (side-by-side) │
├ Results / scriptlogs (bottom) ─────────────────────────────────┐
├ System logs ───────────────────────────────────────────────────┐
└ StatusBar (connection HUD) ───────────────────────────────────┐
+ Studio (Runtime · Wire · Settings) · Command palette (⌘K)
```

Implemented in `sic/app.tsx` (repo root product UI).

AXIS UI: watchlist, multi-pane chart, and editor



Stack

Concern	Choice
Framework	SolidJS 1.9
Bundler	Vite 6 + vite-plugin-solid
Charts	lightweight-charts 5
Editor	CodeMirror 6
Icons	lucide-solid
Style	Tailwind 4 + void tokens (data-theme)

Relation to other tracks

- Operators: [End User](#)
- Structure: [Architecture](#)
- Contracts: [Plugins](#)
- Language: [PYNE runtime](#)

See also

- `LEGACY.md` — what is *not* the AXIS product path
- `TESTING.md` — unit/e2e around UI modules

CHARTING

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED

THIS FILE IS PART OF PYNESCRYPT.

PYNESCRYPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY

IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY

THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR

(AT YOUR OPTION) ANY LATER VERSION.

PYNESCRYPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL, BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF

MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.

YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE

ALONG WITH PYNESCRYPT. IF NOT, SEE

[HTTPS://WWW.GNU.ORG/LICENSES/](https://www.gnu.org/licenses/).

SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Charting" description: "AXIS chart: ChartHost, PaneManager, series factory, drawings, crosshair sync, and trade markers."

CHARTING

Charting is the spatial surface for OHLCV, overlays, and annotations. Implementation is **imperative** under a Solid shell so lightweight-charts owns its DOM subtree.



Abstract

Module	Role
<code>ChartHost.tsx</code> / <code>ChartWorkspace.tsx</code>	Solid mount; multi-slot grid; empty-state
<code>pane-manager.ts</code>	Multi-pane LWC charts, time sync, overlay apply
<code>pane-badge.ts</code>	Corner name + script action icons
<code>series-factory.ts</code>	Candle/volume/line series helpers + palette
<code>chart-type.ts</code>	Price style transforms (HA, hollow, ...)
<code>bar-replay.ts</code> + <code>BarReplayControls</code>	History scrub / play
<code>layout.ts</code> / <code>layout-recipes.ts</code> / <code>chart-registry.ts</code>	Multi-chart layouts + presets
<code>compare-overlay.ts</code>	Second-symbol compare series
<code>volume-profile.ts</code>	OHLCV volume-at-price overlay
<code>drawing-layer.ts</code> + <code>drawings/*</code>	User + script drawings, templates, sync helpers
<code>price-precision.ts</code>	Price-scale decimals (<code>auto</code> or 0–8) from symbol + bars
<code>results/plot-visuals.ts</code>	Pine <code>plot.style_*</code> → LWC series kind (line, stepline, columns, area, ...)
<code>crosshair-sync.ts</code>	Cross-pane crosshair
<code>pyne-drawings.ts</code>	Engine drawing objects → layer
<code>manager-access.ts</code>	Process-wide manager/layer getters

Conceptual model

```
flowchart TB
  HOST[ChartHost Solid] --> EL[panesEl div]
  EL --> PM[PaneManager]
  PM --> PRICE[price pane]
  PM --> VOL[volume pane]
  PM --> IND[indicator pane?]
  PM --> EQ[equity pane?]
  PM --> DL[DrawingLayer]
  RUN[runAndApply] --> PM
  RUN --> DL
  BARS[store.bars] --> PM
```

Interface surface

 Price, volume, and RSI panes with drawing toolbar

Panes

```
store.panes: { id, type, height, order, visible, label }.
```



type	Typical series
price	Price series (style below) + overlay lines + markers + drawings
volume	Histogram
indicator	Non-overlay plots
equity	Strategy equity curve

Defaults: price + volume. Indicator/equity created on demand.

Price chart styles (`store.chartType`)

Topbar **Type** select (persisted). Implementation: `chart/chart-type.ts` + `createPriceSeries`.

chartType	Series	Notes
candles	Candlestick	Japanese candles (default)
hollow	Candlestick	Hollow body on up bars
bars	Bar	Classic OHLC bars
heikinashi	Candlestick	Heikin-Ashi transform of OHLCV
line	Line	Close only
area	Area	Close with fill
baseline	Baseline	Close vs first-bar base

Series still live under pane key `candle` so markers, hlines, and the drawing layer keep a stable host. Changing type swaps the LWC series and rebinds data without a full history reload.

Empty / status overlay

When `bars.length === 0`, `ChartHost` shows contextual title/sub from `store.status` (loading, error, running, or “Load data to begin”).

Price-scale decimals

Price pane labels use `chart/price-precision.ts`:

Mode	Behavior
auto (default)	Merge symbol heuristics (e.g. BTC → 2, SHIB → 8) with recent OHLCV significant digits
0 – 8	Fixed display decimals + matching <code>minMove</code>

UI control cycles **A** → **0** → ... → **8** → **A** on the price-pane scale chrome. Store field: `priceScaleDecimals` (persisted).

Plot style parity

Engine plot series honor Pine Script™ `plot.style_*` via `mapPlotStyleToSeriesKind` (`results/plot-visuals.ts` → pane-manager series factory):

Pine style (examples)	Chart series
<code>plot.style_line</code> / default	Line
<code>plot.style_stepline</code>	Stepline (LWC WithSteps)
<code>plot.style_stepline_diamond</code>	Stepline + vertex point markers
<code>plot.style_histogram</code>	Histogram (base 0)
<code>plot.style_columns</code>	Columns → LWC Histogram (base 0; no column-width/gap API)
<code>plot.style_area</code>	Area
<code>plot.style_circles</code>	Circular point markers + faint hairline
<code>plot.style_cross</code>	Larger point markers, connector hidden
<code>plot.style_linebr</code> / <code>steplinebr</code> / <code>areabr</code>	Broken line / stepline / area — <code>na</code> ends a segment (LWC would otherwise connect through whitespace)

LWC limits: Histogram series has no bar-width or gap option, so `columns` and `histogram` share the same bar geometry (both zero-based). Point markers are always circular — `cross` is approximated by discrete markers without a connector; `stepline_diamond` uses stepline + vertex markers. Sparse `plotshape` styles map diamond/cross to square markers (optional `+` / `x` glyphs when text is omitted).

`fill(plot1, plot2, color=...)` renders as SVG bands between the two series when both resolve on the same pane.

Multi-chart layouts

 2x2 multi-chart grid with BTCUSDT loaded in the active slot

- Modes: **1 / 2H / 2V / 4** (`store.chartLayout`)
- Per-slot symbol/interval via registry; active slot drives topbar + Run
- **Named layouts** save/load (optional chrome snapshot)
- **Recipes** in Layouts menu: Scalp 1m+5m, Swing HTF, Quad, BTC majors (`layout-recipes.ts`)

Bar Replay

 Bar replay strip on BTCUSDT: scrub, play, and speed  Scrubbing and playing bar replay over loaded history

Topbar **Replay** → session over loaded OHLCV (`bar-replay.ts`):

- Enters with **full history visible** (cursor on last bar)
- Scrub / step / play; **Play at end restarts from bar 0**
- Mutually exclusive with Live

Compare & volume profile

 ETHUSDT percent overlay compared against BTCUSDT

 Layers panel: panes, volume profile overlay, scripts, drawings

- **Compare** control: second symbol as % or absolute overlay (`compare-overlay.ts`)
- **Volume profile** toggle in Layers → OHLCV approximation histogram (`volume-profile.ts`)

Pane badges

Each pane's top-left chip (`pane-badge.ts`): label + script actions when scripts are applied:



Icon	Action
Settings	Script inputs modal
File / code	Load that script's source into the docked editor
Eye	Show / hide series
Refresh	Re-run script
Trash	Remove from chart

Volume/equity without scripts: name + hide.

Screenshot

Module-bar **camera** (before Studio) captures Lightweight Charts panes to PNG:

Option	Meaning
Price / Panes / Workspace	Price only, all visible panes stacked, or stacked panes plus a header band
Drawings	Composite the SVG drawing overlay onto the pane
Watermark	AXIS · symbol · interval · time footer
Scale	1× native chart pixels, or 2×
Copy / Download	Clipboard <code>image/png</code> , or <code>axis-SYMBOL-TF-timestamp.png</code>

⌘K → **Screenshot chart...** / **Copy screenshot**. Last options persist in `localStorage`.

Drawings toolbar

See [Drawings](#). Tools include cursor, hline, vline, trend, ray, extend, rect, ellipse, arrow, fib, measure, text (plus fib/shape/annotation/trading packs). The style bar edits color/width/dash plus per-tool extras (extend, fib levels, RR, text); last-used style is stored per kind. Templates and duplicate helpers live under Layers / drawings modules.

Crosshair & time

`syncTimeScales` / `syncCrosshair` keep multi-pane navigation coherent. `scrollToTime` / `jumpToDebugPin` used from Results and debug pins.

Internals

Solid vs imperative boundary

```
// ChartHost – Solid owns outer shell
// PaneManager only mutates panesEl – never put Solid children inside panesEl
```

On cleanup: destroy drawing layer, dispose manager, clear accessors.

Data path

`setDataToChart(bars)` applies OHLCV; `createEffect` on `store.bars` re-applies if bars change outside load helper.



Run apply (chart side)

From `runAndApply` (indicators):

- `syncOverlayLines` — update existing overlay series in place (avoids destroy → blank → add flash on live re-runs)
- Markers from normalized events (set in place)
- Script drawings atomic replace via `DrawingLayer` (`setScriptDrawings` without empty frame); `linefill` quads and `force_overlay` geometry route with other script drawings; `barcolor` tints candles via plot-visuals (not the SVG layer)
- Equity series when strategy report warrants (silent live runs skip hide thrash)

History vs live bars

- Full loads: `loadBars` → `chartDataGen++` → `setDataToChart` + `fitContent`
- Live: `multiplex` → `appendBar` + `manager.appendBar` only (no `fitContent`, no full `setData`)
- **Performance (2.0.x)**: live tick path is O(1) append + conflation under load; heavy history paint is batched; load requests use **AbortController** so symbol switches cancel in-flight venue fetches
- Overlay re-runs prefer in-place series updates (`syncOverlayLines`) to avoid destroy → blank flash

series-factory

Centralizes series construction options, plot-style mapping, and color palette so Results plot summary and chart stay chromatically related.

Invariants

1. Solid reconciliation must not rewrite pane roots.
2. Script drawings ephemeral per run; user drawings store-backed.
3. Overlay routing: engine `meta.overlay` + script type; **oscillator-scale** series forced to sub-pane when they would vanish on price (see [Indicators](#)).
4. Shared non-overlay sub-pane id is stable: `indicator` .
5. Manager may be null before mount—runner no-ops chart apply safely.

Failure modes

Symptom	Cause
Blank panes after hot reload	Manager disposed; full remount
Overlays stack forever	Old path skipped <code>removeOverlays</code>
Fib/tools ignore clicks	No bars / time scale; wrong tool
Markers at t=0	Event times not bar-aligned

Worked example (dev)

```
getManager()?.scrollToTime(barTime);
```



See also

- [Indicators](#)
- [Results and strategy](#)
- [End-user drawings](#)

EDITOR

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED

THIS FILE IS PART OF PYNESCRYPT.

PYNESCRYPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY

IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY

THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR

(AT YOUR OPTION) ANY LATER VERSION.

PYNESCRYPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL, BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF

MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.

YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE

ALONG WITH PYNESCRYPT. IF NOT, SEE

[HTTPS://WWW.GNU.ORG/LICENSES/](https://www.gnu.org/licenses/).

SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Editor" description: "AXIS Pine editor: CodeMirror 6, diagnostics, 80-col ruler, git sync, stats, docked vs popout, editor bridge."

EDITOR

The editor is the **source axis of intent**: Pine text the operator authorizes the engine to run. Completion and hover can use local builtins metadata and optional remote LSP; **evaluation** is always an engine plugin.

 CodeMirror 6 Pine editor showing a v6 RSI indicator



Abstract

Module	Role
<code>EditorPane.tsx</code>	Docked/standalone chrome, header tools, detach
<code>tabbed-editor.tsx</code>	Multi-tab docs, stats strip, Problems, Colors, Symbols, git bar
<code>PyneEditor.tsx</code>	CM6 mount (extensions + view APIs)
<code>pyne-language.ts</code>	StreamLanguage / highlighting (namespaces, types, multiline strings)
<code>pine-symbols.ts</code> + <code>SymbolEmojiManager.tsx</code>	TV-editor-safe glyph catalog + insert panel
<code>pyne-lsp.ts</code> + <code>data/pyne-builtins.json</code>	Offline completion/hover corpus
<code>column-ruler.ts</code>	80-character guide line
<code>diagnostics.ts</code>	Underlines, gutter, jump-to from last run
<code>editor-intel.ts</code>	Settings bag for lint / hover / complete / marks / timings
<code>inline-debug.ts</code>	EOL chips, pin gutter, line flash
<code>git-sync.ts</code> + <code>ui/EditorGitBar.tsx</code>	Pull / push / storage status
<code>ui/EditorProblems.tsx</code>	Collapsible problems list
<code>editor-bridge.ts</code>	Cross-window doc + run messages
<code>cm-void.ts</code>	Void-theme CM styling

Conceptual model

```
sequenceDiagram
    participant Main as Main AXIS
    participant Bridge as editor-bridge
    participant Pop as Popout window
    Main->>Bridge: writeSharedDoc
    Main->>Pop: openEditorWindow
    Pop->>Bridge: hello role=editor
    Main->>Bridge: doc push
    Pop->>Bridge: run
    Bridge->>Main: runAndApply on main
    Main->>Bridge: run-status
```

Why run on main? Chart, bars, and pane manager live in the main window. Popout is a pure editing surface.

Interface surface

Docked mode

- Right dock by default (`panelChrome.editor`); may sit **side-by-side** with other right panels (e.g. Indicators left of Editor)
- FloatableShell: dock left/right/bottom, float, new window
- Detach → popup or tab (`openEditorWindow`)



Popout mode

- Main shows “Editor detached” chip + **Reattach**
- Shared doc key + bridge messages: `hello`, `doc`, `run`, `run-status`, `reattach`, `popout-opened`, `popout-closed`

Run entry points

- Editor / topbar **Run** → `runAndApply(getDoc())`
- Popout Run → bridge → main executes
- Command palette: save library, git push/pull, jump to line, toggles

Header tools (`axis-editor-tools`)

Icon-only (labels in tooltips / `aria-label`). **Open in new tab** is in the panel-icon menu with dock options.

Control	Effect
Run	Run script against loaded bars
Scriptlogs	Open last-run <code>log.*</code> panel
Profiler	<code>profilerEnabled</code> + optional line % gutter; re-run when enabling
Debug	Inline end-of-line log/error chips (<code>inlineDebugEnabled</code>)
Pins	Chart markers +  gutter (<code>debugPinsEnabled</code>); Alt+P
Ruler	80-column guide (<code>editorRulerEnabled</code>)
Panel icon → Open in new tab	Editor in a full browser tab (vs New window popup)

Details: [Debugging](#).

Stats strip

Bottom of tabbed editor (`data-testid="axis-editor-stats"`):

Field	Meaning
Pos L:C	Cursor line:column (1-based)
Ln	Total lines in document
Words / Chars	Document stats
Diagnostics badge	Error/warning count from last run — click jumps to first
Problems	Expand/collapse problems list
Colors	Color chips / picker / converter for hex and <code>color.*</code>
Symbols	Insert TV-editor-safe arrows, box drawing, marks, spaces, and chart emoji (raw, quoted, or <code>plotchar</code>)
wrap	Soft wrap toggle (<code>editorWrapEnabled</code> , default on)

Symbols & emoji

Status-bar **Symbols** (`data-testid="axis-editor-symbols"`) opens a searchable catalog of glyphs used in Pine Script™ `plotchar` / `label.new` / table text.



- **Mono-safe** — ~1 cell in IBM Plex Mono / the TV editor (box drawing, ▲▼◆✓★, arrows).
- **Wide / chart-only** — official ↑↓ and emoji (🔥🚀📈). Filter with **Mono-safe**.
- Click inserts at the cursor (replaces the selection). Right-click copies.
- Insert mode: raw glyph, quoted string, or a `plotchar(cond, "mark", "...", location.belowbar)` snippet.

Sources: TradingView *Exploring Unicode*, [Text and shapes](#), box-drawing U+2500, PineCoders conventions.

Highlighting

`pyne-language.ts` is a stateful stream tokenizer: `//@` annotations, `//` and `/* */` (including across lines), quoted and `""" / '''` strings with `\\n` escapes, `#RRGGBB(AA)` colors, namespaces (`ta.`, `label.new`), types (`int / float / string`, `series / simple` in qualifier position, and declared UDTs / enums as `typeName`, **bold** in the void theme), control vs definition keywords, and function names before `(`. Format leaves continuation lines of an open string or block comment untouched.

Four information surfaces

Surface	When	Answers
Hover	Rest on a symbol	Keyword, type, series, namespace, builtin, user input/function, or hex color
Param checklist	Inside a call	Used / current / unused args with type + default
Pre-eval	While typing / Save	What is wrong <i>before</i> Run
Debug chips	After Run, Debug on	Last-run <code>log.*</code> / errors on that source line

Hover + param checklist are writing-time docs — not a TradingView® debug API. Pre-eval marks the buffer; chips still read **last run**. Details: [Debugging](#).

Diagnostics & Problems

Pre-eval (as you type) — `schedulePreeval` waits the **idle delay** after the last keystroke (default `PREEVAL_IDLE_MS` = 1000 ms; tab switch uses a shorter beat). Marks clear while typing unless **Clear marks while typing** is off. Local parse/lint (`localPreevaluate`) plus `POST /lsp/diagnostics` when engine=server and remote lint is on. Severity **error** disables Run when **Block Run on errors** is on (topbar, editor, Mod-Enter, command palette). Warnings and typos do not block.

Every lint / hover / complete / underline / chip option lives in **Settings** → **Editor** (command palette: *Open Editor Settings*). Defaults match the previous hardcoded behavior.

Local checks: unclosed `( / [ / {` and strings, missing entry point, `study()` → `indicator()` / `strategy()`, bare `security()` → `request.security`, duplicate `indicator()` / `strategy()` / `library()`, unknown builtin members (`strategy.etry`, `plt` → “did you mean ...?”). User functions and import aliases are not flagged.

Merge: remote style-noise C001–C004 is dropped; remote syntax errors win the same span; local unknown-member marks still show when remote is down. Hover tooltips use the diagnostic message (including the suggestion).

After a run, `diagnosticsFromLastRun(lastRun, doc)` still builds ranges from engine `error`, `meta.errors` / `diagnostics`, and error logs with line refs (`line 12:`, `L12:`, ...). Live pre-eval for the current buffer is listed first; last-run engine messages are kept when they add detail (dropped only when the buffer no longer matches the stamped run source).

- CodeMirror: wavy underlines, severity gutter, hover tooltips



- **Problems** panel: clickable rows → `scrollToLine` / `jumpToDiagnostic`; origin **pre-eval** vs **run** when the diagnostic has a source
- Empty / pending pre-eval stays **No problems** (no flicker)
- Unit tests: `tests/editor-diagnostics.test.ts`, `tests/editor-problems.test.ts`, `tests/preevaluate.test.ts`

80-column ruler

`columnRulerExtension` paints a dashed guide at character column **80** (measured via CM coords). Toggle **Ruler** or command palette “Toggle Editor Ruler”.

Git sync bar

Tab toolbar **Pull / Push / status** (`EditorGitBar`):

- Uses active **storage** plugin (`local` | `cloud` | `git`)
- Push → `writeScript` (git commits when storage is git)
- Pull → optional plugin `sync('pull')` + list/reload bound library id
- Config stays in Settings / Script Library (no second credentials UI)

Completion corpus

`src/editor/data/pyne-builtins.json` is synced from PYNE (`scripts/sync-pyne-builtins.sh`). Offline completion/hover fall back to this catalog; optional remote LSP when the Pro API is available.

Library load

`loadLibraryDoc(doc, name)` on `editorRef` injects storage reads into the active tab.

Internals

editorRef pattern

App holds `{ getDoc, setDoc?, loadLibraryDoc?, scrollToLine?, jumpToDiagnostic?, getCursor?, insertAtCursor? }` populated by CM mount—avoids full-doc store writes every keystroke (doc also mirrored to `EDITOR_DOC_KEY`).

Persistence

Key / field	Role
<code>store.editor</code> + <code>panelChrome.editor</code>	Dock geometry, open, mode
<code>pynescrypt.axis.editor.doc</code>	Draft text
<code>profilerEnabled</code> / <code>inlineDebugEnabled</code> / <code>debugPinsEnabled</code> / <code>editorRulerEnabled</code> / <code>editorWrapEnabled</code>	Editor tools

Invariants

1. Empty doc Run is a no-op at topbar/editor handlers.
2. Popout does not own `PaneManager`.



3. Reattach restores shared doc into docked CM.
4. AXIS $\neq$ engine: editor never fully evaluates Pine itself; pre-eval is parse/lint only.
5. Debug/Pins still read **last run**; static diagnostics also come from pre-eval.

Failure modes

Symptom	Mitigation
Two editors diverge	Prefer bridge doc events; reattach
Detached but no window	Popup blocked; allow popups; reattach
Lost text on crash	editor.doc localStorage; library Save / git push
Blank CM (no lines)	Dock column height chain; editor must fill strip
No chips/pins	Enable Debug/Pins; logs need line and/or bar_index

See also

- [Debugging](#) — Debug vs Pins, Scriptlogs, Profiler
- [UI shell](#) — Topbar, docks, command palette
- [Indicators](#)
- [Script library](#)

DEBUGGING

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED

THIS FILE IS PART OF PYNESCRYPT.

PYNESCRYPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY

IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY

THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR

(AT YOUR OPTION) ANY LATER VERSION.

PYNESCRYPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL, BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF

MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.

YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE

ALONG WITH PYNESCRYPT. IF NOT, SEE

[HTTPS://WWW.GNU.ORG/LICENSES/](https://www.gnu.org/licenses/).

SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Debugging" description: "Scriptlogs, Profiler, Debug chips, chart Pins, diagnostics, and System Logs."

DEBUGGING

AXIS separates **script-authored diagnostics** (Pine `log.*`, engine errors, line timing) from **shell telemetry** (load, stream, engine connection).

 Script Logs panel for last-run Pine log output

Writing-time and last-run info share four surfaces — hover, the param checklist, pre-eval, and debug chips — so each answer stays on its own chrome.



Abstract

Surface	Source of truth	Open / toggle
Hover	Builtin / identifier docs (local corpus or LSP)	Rest cursor on a symbol
Param checklist	Call signature + remaining named args	Type inside <code>plot(/ ta.sma(/ ...</code>
Pre-eval / Problems	Parse/lint u last-run engine errors	Underlines + Problems (<code>pre-eval</code> vs <code>run</code>)
Scriptlogs	<code>store.lastRun</code> → <code>normalizePyneLogs</code>	Editor header Scriptlogs
Profiler	<code>profilerEnabled</code> + <code>profile.lines</code>	Editor header Profiler
Debug chips	<code>inlineDebugEnabled</code> + last-run logs with <code>line</code>	Editor header Debug
Pins	<code>debugPinsEnabled</code> + logs with <code>bar_index / time</code>	Editor header Pins · Alt+P
System Logs	<code>store.logs</code> ring buffer	Status bar / logs strip

Module	Role
<code>ui/ScriptLogsPanel.tsx</code>	Floatable last-run <code>log.*</code> list
<code>results/pyne-logs.ts</code>	Normalize / filter / TSV
<code>results/profiler.ts</code>	Line % profile
<code>results/inline-debug.ts</code>	Annotations + pinable helpers
<code>results/debug-pins.ts</code>	Bar pins, markers, <code>countDebugPins</code>
<code>editor/inline-debug.ts</code>	CM chips, pin gutter, line flash
<code>editor/diagnostics.ts</code>	Error underlines / gutter / jump
<code>ui/EditorProblems.tsx</code>	Problems list UI
<code>chart/manager-access.ts</code>	<code>applyDebugPinsToChart</code> , <code>jumpToDebugPin</code>

Conceptual model

```
flowchart LR
    SRC[Editor buffer] --> PRE[Pre-eval]
    PRE --> DIAG[Diagnostics / Problems]
    PIN[Pine log.* / errors] --> ENG[Engine run]
    ENG --> RR[lastRun]
    RR --> SL[Scriptlogs]
    RR --> DBG[Debug chips]
    RR --> PINZ[Pins + markers]
    RR --> DIAG
    RR --> PR[Profiler gutters]
    APP[Load / stream / boot] --> SYS[System Logs]
```

Invariant: Scriptlogs / Debug / Pins never write into `store.logs` .



Four information surfaces

Surface	When	Answers
Hover	Idle on a symbol	What is this builtin / identifier?
Param checklist	Inside a call	Which arguments remain?
Pre-eval	While typing / Save	What is wrong <i>before</i> Run?
Debug chips	After Run, Debug on	What did this source line log or error?

Hover and the param checklist never invent TradingView® host APIs — they read PYNE builtins / local docs. Pre-eval marks the buffer; Debug chips still read **last run** only.

Debug vs Pins

	Debug	Pins
Question	What happened on this source line ?	Which bar was that on the chart?
Requires	Line ref (<code>line 12:</code> , structured <code>line</code> ,...)	<code>bar_index</code> and/or <code>time</code> / <code>bar_time</code>
UI	End-of-line chips + optional level gutter	Chart circle markers +  editor gutter
Click	Pin-able chips jump if bar info present	Chip /  / Scriptlogs → chart jump + line flash
Store	<code>inlineDebugEnabled</code>	<code>debugPinsEnabled</code>

Workflow

1. Add `log.info` / `log.warning` / `log.error` (or rely on engine errors) with **line** and ideally **bar_index**.
 2. **Run**.
 3. Toggle **Debug** to see chips; **Pins** to mark bars (count shows on the Pins button).
 4. Click a pin-able chip or  → `jumpToDebugPin` (crosshair + scroll); source line flashes (`cm-debug-pin-flash`).
- Both can be on at once. Pins without Debug still show  gutter and chart markers.

Scriptlogs

Panel for **script** logging from the last run.

Pine call	Panel level
<code>log.info</code>	info
<code>log.warning</code>	warning
<code>log.error</code>	error

Open: Editor → **Scriptlogs** (`axis-btn-scriptlogs`). Pin-able rows jump to the chart when bar time/index is known.



Editor Profiler

Toggle **Profiler** (`axis-btn-profiler`):

- Persists `profilerEnabled`
- May send `profiler: true` to the engine
- Header shows last-run ms
- Gutter shows line % when `profile.lines` is present

Diagnostics & Problems

Independent of Debug chips: structural **error reporting** from pre-eval and the run payload.

- Pre-eval (`preeval*`) marks parse / lint issues as you type; last-run adds engine `error` , `meta.errors` , `diagnostics` , error-level logs
- CodeMirror underlines + severity gutter (larger hit target than line numbers)
- Stats strip badge (`axis-editor-diag-count`) and **Problems** list (`axis-editor-problems`)
- Each row shows origin `pre-eval` or `run` when the diagnostic has a source
- Click → jump to line/range
- Empty / in-flight pre-eval stays **No problems** (no checking flicker)

Scriptlogs vs System Logs

	Scriptlogs	System Logs
Source	Script <code>log.*</code> on last run	AXIS shell events
Store	<code>lastRun</code>	<code>store.logs</code>
UI	Floatable Scriptlogs	Strip above status bar
Control	Editor header	Status bar

Persistence & test ids

Key	Role
<code>panelChrome.scriptlogs</code> / <code>editor</code>	Panel chrome
<code>profilerEnabled</code>	Profiler
<code>inlineDebugEnabled</code>	Debug chips
<code>debugPinsEnabled</code>	Chart + pin gutter



testid	Role
axis-btn-scriptlogs	Open Scriptlogs
axis-btn-profiler	Profiler
axis-btn-inline-debug	Debug
axis-btn-debug-pins	Pins
axis-debug-pin-count	Pin count
axis-editor-diag-count	Diagnostics badge
axis-editor-problems	Problems panel

Unit coverage: tests/pyne-logs.test.ts , tests/profiler.test.ts , tests/inline-debug.test.ts , tests/debug-pins.test.ts , tests/editor-diagnostics.test.ts .

See also

- [Editor](#)
- [Charting](#) — markers and jump
- [UI shell](#)

UI SHELL

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED

THIS FILE IS PART OF PYNESCRYPT.

PYNESCRYPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY

IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY

THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR

(AT YOUR OPTION) ANY LATER VERSION.

PYNESCRYPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL, BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF

MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.

YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE

ALONG WITH PYNESCRYPT. IF NOT, SEE

[HTTPS://WWW.GNU.ORG/LICENSES/](https://www.gnu.org/licenses/).

SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "UI shell" description: "AXIS chrome: topbar, watchlist, status bar, settings, plugin manager, logs, and layout chrome around the chart."

UI SHELL

The shell is the non-canvas chrome: navigation of axes, market selection, modals, and status. It binds human gestures to store + plugins without owning calculation.

 **AXIS topbar groups: market, venue, engine, panels, Studio**  **Command palette filtering theme commands**



Abstract

Component	Path	Role
Topbar	ui/Topbar.tsx	Axes pickers, Load/Run/Live, theme, modals
Watchlist	ui/Watchlist.tsx	Symbols, quotes, click-to-load
StatusBar	ui/StatusBar.tsx	Status, engine/storage badges, trade snippet
StudioHost	ui/studio/StudioHost.tsx	Full-page Runtime / Wire / Settings / Workers / Plugins
SettingsPage	ui/settings/SettingsPage.tsx	Appearance, chart, live, keys, editor, theme
PluginsPage	ui/plugins/PluginsPage.tsx	Catalog / install / library
WirePage	ui/wire/WirePage.tsx	Compose-recipe wiring (source × stream × engine × storage × dataset)
SystemLogs	ui/SystemLogs.tsx	Ring buffer of store.logs
ResultsPanel	ui/ResultsPanel.tsx	Bottom drawer (see results doc)
Icons	ui/icons.tsx	Lucide wrappers
ResizeHandle	ui/ResizeHandle.tsx	Panel widths/heights

Conceptual model

```
flowchart LR
    TOP[Topbar] --> AP[setActivePlugin]
    TOP --> LOAD[loadSymbolData]
    TOP --> RUN[runAndApply]
    TOP --> LIVE[startLive/stopLive]
    WL[Watchlist] --> LOAD
    SET[Settings] --> AP
    PM[PluginManager] --> REG[registry]
    PM --> LIB[storage service]
    SB[StatusBar] --> ST[store.status]
```

Topbar responsibilities

Grouped left → right (axis-tb-group , data-tb-group):

Group	Contents
brand	Logo + AXIS
market	Symbol, Interval, Type (TopbarField), Compare
data	Source (+ CSV upload), Load, Reload (icon-only)
compute	Engine, Stream, Run, Live, Replay
layout	Multi-chart layouts + recipes
panels	List, Editor, Library, Data (Source Manager), Scripts, Layers, Alerts, Data window, Inputs, Results
system	Second-row trailing: theme, fullscreen, chart-only, screenshot, Studio



Control	Effect
Symbol / interval	Store + Load (Enter / blur reload)
Source / Stream / Engine	<code>setActivePlugin</code> + side effects
Chart type	<code>setChartType</code> (candles, HA, ...)
Compare	Second-symbol overlay
Load / Reload	One-shot historical fetch → chart
Data panel	Data Source Manager — background deep history
Scripts panel	Applied indicators/strategies list
Run	Editor doc → <code>runAndApply</code> ; accent only while <code>status === 'running'</code>
Live	multiplex start/stop (stops Replay)

Chart themes

Settings / command palette expose **ten** curated presets (void dark/light, classic, mono, obsidian, graphite, pacific, dusk, porcelain, parchment). High-end soft surfaces — not neon high-contrast. See `src/theme/presets.ts` . | Replay | Bar replay over loaded bars | | Panel toggles | `isPanelOpen` / dual-write chrome | | Settings / Plugins | Modal open | | Detach editor | Bridge + popout |

Integrated labels: `ui/TopbarField.tsx` (`.axis-tb-field*`).
`catalogTick` forces memo re-list when plugins install/remove.

Command palette

Ctrl/Cmd+K — `CommandPalette` + `command-registry.ts` : panels, theme, layouts, Run, symbol focus, editor toggles (ruler, debug, pins, profiler), jump to line, git push/pull, save library. Palette entries render any recorded shortcut chords.

Keyboard shortcuts

Global chord dispatch hub (`src/ui/shortcuts/`): capture-phase handling with a **dialog-skip guard** so shortcuts never fire while a modal is open. Wired to three scopes:

- **App-level** — palette, Run, panel toggles, replay, layouts
- **Chart** — keymap routed through the dispatch hub (cancel draft, layouts, replay)
- **Editor** — `src/editor/cm-line-ops.ts` `Prec.high` line ops: duplicate line, toggle comment, indent/unindent, delete line

Recorder and discovery:

- **Recorder**: Settings → Keyboard → record a chord; per-user overrides persist on the app store
- **Shortcuts modal**: lists default bindings with live rendering
- **Conflict flags**: scope-aware (`app.escape` vs `chart.cancel-draft` layer by design — no false conflict)

Alerts panel

Dockable **Alerts** (`src/alerts/*` + `AlertsPanel.tsx`): local-first book for **price, % change, drawing touch, indicator plots, Pine `alert()` / `alertcondition()`** , and on-chain kinds. Delivery: browser Notification, optional HTTPS webhook, optional L2 webhook. Live ticks evaluate price/drawing/plot kinds; strategy/indicator runs evaluate Pine `alert()` events (last bar). Continuous server-side arming is not required for session use.

Workspace snapshots

`storage/workspace-snapshot.ts` + optional chrome menus capture layout/plugin prefs for restore (distinct from OHLCV — bars still re-fetch on Load).

Panel docks (side-by-side)

Left/right docks with **2+ open panels** lay out **horizontally** (row), not stacked under each other:

- Example: **Indicators** left of **Editor** on the right strip
- Column width = **sum** of panel widths (capped)
- Each panel keeps its own width (resize handle on chart-facing edge)
- Bottom dock still stacks vertically

See `ui/panels/dock-layout.ts`, `FloatableShell.tsx`.

Mobile shell (phones / tablets)

`src/ui/mobile/MobileShell.tsx` + `src/ui/responsive.ts` — reactive `phone | tablet | desktop` mode from viewport + `pointer: coarse` (desktop $\geq 1024\text{px}$ is unchanged):

- **Phones ($\leq 767\text{px}$):** compact header (brand · venue · symbol · interval) + bottom tab bar (**Chart · Panels · Editor · Studio · More**) replace the Topbar/StatusBar. Panels and More open as **nav overlays**.
- **Panel sheets:** docked/floating panels render as full-viewport sheets — one at a time, dismiss via **swipe-down** on the title bar. Desktop panel geometry is bypassed, never overwritten, so phone ↔ desktop switches preserve layouts. Phones boot chart-first.
- **Force-single chart:** 2H/2V/4 layouts collapse to the active slot with a slot-switcher chip row; other slots stay preserved in `store.chartLayout`.
- **Tablets (768–1023px):** desktop shell with a horizontally scrollable topbar.
- Dialogs become **bottom sheets**; safe-area insets, `100dvh` root, $\geq 44\text{px}$ touch targets.

Watchlist



Watchlist rail with BTCUSDT selected and live 24h quotes

- Default universe includes majors (`BTCUSDT` , ...)
- Named lists persist (`watchlist.lists` + `activeId`); legacy `symbols` -only state migrates to a **Main** list
- Create / rename / duplicate / delete lists from the panel header or the panel-icon menu
- Row bell creates a price alert (cross / above / below) at last
- `refreshSec` clamped 5–120
- Source-aware labels (e.g. CSV mode)
- Click sets symbol and loads through active source

Status bar



Connection HUD: LIVE, tick, engine, and source/stream/storage chips

Single row:

- **Left — Connection HUD** (`ConnectionHud` , `data-testid="axis-connection-hud"`)
 - LIVE badge (off / connecting / live / err)



- Tick pulse (fixed-width price + age; ping does not resize layout)
- Engine chip (id · interpret/compile · latency)
- Plane chips SRC / STR / ENG / STO with transport badges **WS / REST / LOCAL / BROKER**
- Source ↔ stream pairing warning when mismatched
- **Right** — `status` / `statusMessage`, strategy snippet, log toggle, bar/ind/pane counts

Telemetry is ephemeral (`store.telemetry`); HUD compact mode and live prefs persist via settings.

Live settings (Settings dialog)

Setting	Default	Effect
Enable live stream	on	<code>live.preferAfterLoad</code> → multiplex after Load and on boot when bars exist
Indicator re-run	every-tick	or <code>bar-close</code> only
Compact connection HUD	off	hide plane chips

Logs

System Logs strip

- `appendLog(level, message, source)`
- Cap `MAX_LOGS` (500)
- Not persisted
- Boot messages, pyodide ready, live errors

Scriptlogs & Profiler

Script `log.*` output and editor line-timing live on a separate path — see [Debugging](#). Controls sit on the **editor header** (not the topbar) and do not write into `store.logs`.

Theming

 Settings → Theme: Void Dark preset and chart color tokens  Switching chart theme presets

`store.theme` → `document.documentElement data-theme` (`dark` | `light`). Void dark is default product aesthetic. Settings → **Theme** picks a curated preset (void / classic / mono / boutique) and edits chart tokens live.

Accessibility / test hooks

E2E selectors use `data-testid` on critical controls (`axis-btn-load`, `axis-manager`, ...) per `TESTING.md`.

Invariants

1. Shell never calls Python.
2. Plugin lists always come from registry facades (`listSources`, ...).
3. Modal settings persist only on Save (not on every keystroke of endpoint field until save).
4. Live off leaves historical bars intact.



Failure modes

Symptom	Shell-side check
Pickers empty	Builtins not registered at boot
Live ignores click	Live is off (<code>stream: null</code>) or a multiplex error is in the logs
Settings probe fails	Network / CORS—not topbar bug

See also

- [Manager and settings](#)
- [Store](#)
- [Architecture overview](#)

INDICATORS

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED

THIS FILE IS PART OF PYNESCRYPT.

PYNESCRYPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY

IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY

THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR

(AT YOUR OPTION) ANY LATER VERSION.

PYNESCRYPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL, BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF

MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.

YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE

ALONG WITH PYNESCRYPT. IF NOT, SEE

[HTTPS://WWW.GNU.ORG/LICENSES/](https://www.gnu.org/licenses/).

SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Scripts (indicators & strategies)" description: "AXIS applied-scripts UI: runAndApply pipeline, Scripts panel, overlay routing, and engine binding."

SCRIPTS (INDICATORS & STRATEGIES)

In AXIS, **scripts applied to the chart** may be Pine `indicator()` or `strategy()` definitions. The product panel is labeled **Scripts** (panel id remains `indicators` for workspace compatibility). The module boundary is `indicators/runner.ts` plus list UI—not the PYNE evaluator.



Scripts panel listing an applied RSI indicator



Abstract

Piece	Role
<code>runScript</code>	Engine invocation + status/timeouts
<code>runAndApply</code>	Chart + lastRun + results open; may <code>addIndicator</code>
<code>probeEndpoint</code>	Settings health check helper
Scripts panel (<code>IndicatorPanel</code>)	List / toggle / colors for <code>store.scripts</code>
<code>IndicatorCard</code>	Per-script UI chrome

Language semantics: [PYNE runtime](#).

Conceptual model

```
flowchart TB
    DOC[Pine source] --> RS[runScript]
    BARS[store.bars] --> RS
    CFG[getActiveEngineConfig] --> RS
    RS --> ENG[EnginePlugin.run]
    ENG --> RR[RunResult]
    RR --> LR[setLastRun]
    RR --> CH[PaneManager overlays]
    RR --> MK[Trade markers]
    RR --> DR[Script drawings]
    RR --> RES[Results open]
```

Interface surface

runScript options

Option	Default	Meaning
<code>silent</code>	false	Quiet status; live re-run logging
timeout	derived	Interactive: scales with bars (60–180s); silent 45s

runAndApply options

Option	Default	Meaning
<code>openResults</code>	<code>!silent</code>	Open results drawer
<code>indicatorId</code>	optional	Update existing store indicator

Overlay routing

```
overlay = resolveOverlayFlag(meta.overlay, script_type)
// explicit false → sub-pane
// explicit true → price
// missing: indicator → sub-pane; strategy → price

if overlay && seriesWouldHideOnPrice(plots, bars):
  overlay = false // RSI / scaled oscillators on BTC scale
  paneId = overlay ? 'price' : `ind_<scriptId>` // one sub-pane per script
```

Each non-overlay script gets its own sub-pane (`ind_<id>`), not a shared `indicator` host. Re-runs keep the script's private pane; legacy shared `indicator` panes migrate on re-run. Removing a script clears only that owner's series and destroys the sub-pane only when empty.

Persist + reopen

Applied scripts (`store.scripts` with full `code` , `paneId` , colors, inputs, strategy props) are written to `localStorage` with the rest of durable UI state. On boot they hydrate via `sanitizePersistedScripts` . After each successful history paint (`load-symbol` , DSM cache apply), `reapplyChartScripts` silently re-runs every visible script in plot-dependency order so the chart matches the saved set without manual re-run.

Detach path: `detachIndicatorFromChart` (Scripts panel, Layers, pane badges) — owner-scoped overlay clear, orphan pane sweep, no sibling wipe.

Prefer `indicator(..., overlay=false)` and raw `plot(rsi)` for oscillators — not `strategy(..., overlay=true)` with `rsi * 0.01` on price.

Series selection

Prefer `result.series` entries whose keys do not start with `_` / `__` . Fallback: single `plots[]` array as one line named from `script_name` .

Colors: `plot_meta[k].color` or `PLOT_PALETTE[i]` . Pine Script™ `plot.style_*` maps to LWC series kinds (line, stepline, histogram/columns, area, circles) — see [Charting — plot style parity](#).

Indicator panel & pane badges

- Side list of `store.scripts` (`Indicator` : id, name, code, paneId, visible, plots)
- Visibility toggles affect display; re-run is still engine-driven
- Chart pane chips mirror settings / load source / eye / re-run / remove (see [Charting](#))

Active engine binding

```
// plugins/active.ts pattern
getActiveEngine() // registry lookup by activePlugins.engine
getActiveEngineConfig() // pluginsConfig + store.endpoint merge
```

Switching engines mid-session does not auto-rerun; operator hits Run.



Live re-run coupling

Streams may set `live.needsRerun` ; multiplex/runner path can call `runAndApply` with `silent: true` so the status bar is not spammed. Failures go to logs.

Internals

Path	Role
<code>src/indicators/runner.ts</code>	Pipeline
<code>src/indicators/IndicatorPanel.tsx</code>	List UI + built-in picker
<code>src/indicators/builtins/</code>	Original AXIS built-in Pine catalog
<code>src/plugins/active.ts</code>	Active engine/source helpers
<code>src/engines/catalog.ts</code>	Built-in engines
<code>src/results/plot-visuals.ts</code>	Plot style → series kind

Invariants

1. Runner is the **only** chart-facing calc orchestrator (avoid duplicate apply logic in components).
2. Errors still `setLastRun` with `status: 'error'` when possible.
3. Clearing overlays happens before re-adding—no unbounded series leak.
4. AXIS does not interpret Pine; it only maps `RunResult` .

Failure modes

Symptom	Layer
Engine exception	Results error + status
Success empty plots	Script/meta—not runner
Manager null	Chart not mounted; run still sets lastRun
AbortError	Timeout—reduce bars or optimize script

See also

- [Charting](#)
- [Results and strategy](#)
- [Built-in scripts](#)
- [ADR-002 / ADR-004](#)

RESULTS AND STRATEGY

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED

THIS FILE IS PART OF PYNESCRYPT.

PYNESCRYPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY

IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY

THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR

(AT YOUR OPTION) ANY LATER VERSION.

PYNESCRYPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL, BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF

MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.

YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE

ALONG WITH PYNESCRYPT. IF NOT, SEE

[HTTPS://WWW.GNU.ORG/LICENSES/](https://www.gnu.org/licenses/).

SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Results and strategy" description: "UI-side event normalization, trade pairing, metrics, equity curve, and ResultsPanel export pipeline."

RESULTS AND STRATEGY

Results UI turns opaque engine payloads into **operator-consumable tables, metrics, and files**. It is a pure-ish viewer layer over `lastRun` + `bars`.



Results overlay with strategy stats, equity curve, and closed trades



Abstract

Module	Role
results/events.ts	Normalize heterogeneous event shapes; markers; equity series
results/strategy.ts	Shared walker <code>walkStrategyEvents</code> → trades, stats, positions, stream; CSV
results/positions.ts	Pyramiding-aware position cycles (avg-entry parity with the pyne broker, per-fill P&L)
ui/ResultsModal.tsx	Tabs, Events Stream/Open⇌Close switcher, export, jump-to-time
ui/StrategyReport.tsx	Stats cards (return %, bars-held, fills), equity curve, trades table, CSV
ui/StatusBar.tsx	Compact trade summary

Conceptual model

```
flowchart LR
  LR[lastRun.events] --> N[normalizeStrategyEvents]
  B[store.bars] --> N
  N --> T[buildStrategyReport]
  N --> M[eventsToMarkers]
  N --> E[buildEquityCurve]
  T --> UI[Strategy tab]
  LR --> P[Plots / Metrics / Raw]
  T --> CSV[trades CSV]
  LR --> JSON[run JSON]
```

Event normalization

Engines may emit Pro API parity fields (`kind` , `bar_time` , `direction` , `ohlc`) or legacy UI fields (`type` , `time` , `dir` , `price`). Normalizer:

- Unifies time/price/dir/id
- Optionally fills price from bars when OHLC present/empty
- Can include order-like events for the Events tab

Trade pairing algorithm

Sketch (`buildStrategyReport`):

1. Sort by time.
2. On entry-like kind → open map by `id` .
3. On exit/close-like → match id, else sole open trade.
4. $PnL = \Delta price \times long/short \text{ sign}$.
5. Aggregate win rate, profit factor, max DD on cumulative PnL.

Not modeled: fees, funding, partial fills, portfolio multi-symbol—viewer honesty.



ResultsPanel tabs

Tab	Data source
Events	<code>normalizedEvents</code> — Stream (chronological) or Open↔Close (position cycles via <code>results/positions.ts</code>) view
Strategy	<code>report.trades</code> + <code>report.stats</code> (single column: stats → equity → trades)
Plots	<code>series</code> / <code>plots</code> summary
Metrics	meta + context + stats subset
Raw	<code>JSON.stringify(lastRun)</code>

Interactions: copy, export, `scrollToTime` on trade rows. Scriptlogs / Debug **Pins** also jump the chart (`jumpToDebugPin`) when bar time/index is present — see [Debugging](#).

Equity

`buildEquityCurve` feeds chart equity pane when runner/strategy path requests it—derived series, not engine-native unless also provided.

Export (ADR-013)

Artifact	Function
<code>axis-run-*.json</code>	Full payload
<code>axis-trades-*.csv</code>	<code>tradesToCsv</code>
Clipboard	CSV when permitted

Invariants

1. Results recompute from `lastRun` reactively; they do not re-query engines.
2. Changing bars without re-run can change normalization fills—re-run after Load.
3. `lastRun` not rehydrated from localStorage (session artifact).
4. Infinite profit factor when no losses but profits exist—display must tolerate non-finite.

Failure modes

Symptom	AXIS explanation
Events > 0, trades = 0	Unpaired or missing prices
CSV disabled	No closed trades
Jump no-ops	Manager unmounted / invalid time

Internals cross-links

Operator narrative: [Strategy and results](#).

Architecture: [ADR-013](#).



See also

- [Indicators](#)
- [Charting](#)

STORE

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED

THIS FILE IS PART OF PYNESCRYPT.

PYNESCRYPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY

IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY

THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR

(AT YOUR OPTION) ANY LATER VERSION.

PYNESCRYPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL, BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF

MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.

YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE

ALONG WITH PYNESCRYPT. IF NOT, SEE

[HTTPS://WWW.GNU.ORG/LICENSES/](https://www.gnu.org/licenses/).

SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Store" description: "AXIS Solid store: AppState shape, activePlugins, persistence rules, logging, and alignment helpers."

STORE

The Solid store is the **control-plane nexus** of AXIS. Plugins read configuration from it; UI binds reactively; persistence snapshots a safe subset to `pynescrypt.axis.v1`.



Abstract

Export	Role
store / setStore	createStore<AppState>
persist	Debounced localStorage write
setActivePlugin	Kinded selection + flat field mirrors
appendLog / status helpers	Operator-visible telemetry
STORAGE_KEY	pynescrypt.axis.v1
MCP	src/mcp/dispatch.ts reads/writes allowlisted store paths; snapshots redact secrets

Types: src/store/types.ts .

Conceptual model

```
flowchart TB
    UI[Solid components] -->|setStore| S[store]
    S -->|persist subset| LS[(localStorage)]
    LS -->|loadPersisted| S
    S --> ENG[engine config readers]
    S --> SRC[source/stream readers]
    S --> CH[chart effects]
```



AppState map

Field	Persistence	Notes
bars	no	Session history
symbol , interval , exchange	yes	Market context
source , engine	yes	Flat mirrors
endpoint	yes	Server engine base URL
activePlugins	yes	Canonical four-tuple
pluginsConfig	yes	Per-plugin fields
scripts , panes	yes	Indicator list + layout
live	partial	streamId important
theme	yes	dark/light
editor , watchlist , panels	yes	Chrome geometry
stream.status	yes/volatile	Connection hint
status , statusMessage , lastRunMs	yes*	UX; ok to restore
lastRun	no	Forced null on hydrate
logs	no	Forced empty
profilerEnabled	yes	Editor profiler (line % when engine provides profile)
inlineDebugEnabled	yes	End-of-line Debug chips from last-run logs
debugPinsEnabled	yes	Chart pin markers + editor pin gutter
editorRulerEnabled	yes	80-col guide (default on)
editorWrapEnabled	yes	Soft line wrap (default on ; stats strip wrap toggle)
panelChrome / alertsPanel / layerPanel / dataViewPanel	yes	Dock chrome (side-by-side left/right)
chartLayout / savedLayouts	yes	Multi-chart grid + named layouts
compare	prefs yes	Second-symbol overlay (bars ephemeral)
drawingTool	reset cursor	On hydrate
drawings / drawingPrefs / drawingUi	yes	User annotations + toolbar prefs
selectedDrawingId	no	Ephemeral selection

*Status fields may restore but boot also appends fresh logs.

Defaults (product)

Representative defaults from store:



Key	Default
symbol	BTCUSDT
interval	1d
source / active source	binance-rest
stream	binance-ws
engine	server
storage	local
endpoint	demo/API host or localhost depending on build era
editor width	460 docked open
watchlist	majors on Main , 15s refresh; named lists persist

Treat demo endpoints as **overridable**—operators should set local URLs in desk topology.

setActivePlugin

```
setActivePlugin(kind, id):
  activePlugins[kind] = id
  if source → source = id
  if engine → engine = id
  if stream → live.streamId = id
  persist()
```

Prevents registry/store split brain ([architecture overview](#)).

Logging

```
appendLog(level, message, source?)
levels: info | ok | warn | error
MAX_LOGS = 500
```

`setStatus` maps app status to log levels for important transitions.

Legacy dual path

Vanilla `state.js` remains for legacy shell with the same storage key family. Product AXIS uses Solid store only—do not dual-write new features to `state.js` unless maintaining legacy.

Invariants

1. Hydrate never restores `lastRun` or `logs`.
2. Persist strips `bars`.
3. Key migration is read-repair ([state namespaces](#)).
4. `pluginsConfig` keys prefer ``${kind}:${id}``.
5. Store is not a plugin registry—ids must exist in registry to function at runtime.



Failure modes

Mode	Behavior
JSON parse fail	Defaults
localStorage throws	No-op persist
Unknown plugin id	UI may show id; run/load fails at getActive*
Oversized drawings	Quota risk—export/clear

Testing notes

- Import `tests/setup.ts` for localStorage stubs
- Unit coverage includes store + migration patterns
- Do not require real IDB in pure store tests

See also

- [State namespaces](#)
- [ADR-011](#)
- [UI shell](#)